

**Unit – III****TWO Marks Questions****1. List any two controls with their purpose.**

- **Label:** Labels provide text information and are defined with class Label. A Label displays text that the user cannot directly modify.
- **TextBox:** A textbox is an area in which either text can be displayed by a program or the user can type text via the keyboard.

**2. What is a Windows Form?**

Windows Forms are used to create the GUIs for programs. A Form is a graphical element that appears on your computer's desktop; it can be a dialog, a window or an MDI window. The controls and components are organized into categories by functionality. Selecting the category All Windows Forms at the top of the Toolbox allows you to view all the controls and components from the other tabs in one list

**3. What is Toolbox? List any two control categories.**

**ToolBox** in **C# .NET (within Visual Studio)** is a panel or window that provides a collection of **controls** and **components** that developers can drag and drop onto a **Windows Form** or **WPF/XAML** designer. These controls are used to build the user interface of applications.

**Two Control Categories in C# .NET:****a) Common Controls**

These are frequently used UI elements, such as:

- a. Button      b. Label      c. TextBox      d. CheckBox      e. RadioButton

**b) Containers**

Controls used to hold and organize other controls:

- a. GroupBox      b. Panel      c. TabControl      d. SplitContainer

**4. Write the purpose of Hide and Show methods of the Form.****Show() Method**

- The Show() method is used to **display the form** on the screen.
- When you call this method, the form becomes visible to the user.

**Code:**

```
Form2 f2 = new Form2(); // create an object of Form2
f2.Show();               // call built-in Show() method
```

**Hide() Method**

- The Hide() method is used to **make the form invisible** without closing it.
- Unlike Close(), the form object still exists in memory; it is just not visible to the user.

**Code:**

```
Form2 f2 = new Form2(); // create an object of Form2
f2.Hide();               // call built-in Show() method
```

**5. What is the common event of the Form? When does it occur?**

C# Windows Forms, the most common event of a Form is the Load event.

**Form Load Event**

- Event Name: Form.Load
- When it occurs:  
It occurs when the form is loaded into memory and is about to be displayed for the first time.
- This is the place where you usually write initialization code (e.g., setting default values, loading data from a database, adjusting control properties).

Example:

```
private void Form1_Load(object sender, EventArgs e)
{
    MessageBox.Show("Form is loaded and ready!");
}
```

**6. What is a Label control? Give any two of its properties with purpose.****Label Control in C#**

A **Label** control in Windows Forms is used to **display text** on a form.

- It is mainly for showing **static text, instructions, headings, or descriptions**.
- A label is **read-only** (users cannot edit it like a TextBox).

**Two Common Properties of Label:****1. Text**

- **Purpose:** Specifies the text that the label displays.
- Example:
- `label1.Text = "Enter your Name:";`

**2. ForeColor**

- **Purpose:** Sets the **color of the text** displayed in the label.
- Example:

```
label1.ForeColor = Color.Red;
```

## 7. How to make a TextBox as password textbox?

In **C# Windows Forms**, you can make a **TextBox** work like a **Password TextBox** (where typed characters are hidden) by using the **PasswordChar** or **UseSystemPasswordChar** property.

### 1. Using PasswordChar

- This property lets you specify a character (like \* or •) that will be shown instead of the actual text.

```
textBox1.PasswordChar = '*';
```

Now, if the user types hello, it will appear as \*\*\*\*\*.

### 2. Using UseSystemPasswordChar

- When set to true, the TextBox automatically uses the system's default password character.

```
textBox1.UseSystemPasswordChar = true;
```

This usually shows a **dot (•)**, like in most login forms.

## 7. How to make a TextBox as password textbox?

To make a TextBox behave like a password textbox in C# .NET, you need to mask the characters typed by the user. This is done by setting the PasswordChar property (in Windows Forms) or using PasswordBox (in WPF).

### In Windows Forms:

Use the PasswordChar property of the TextBox control:

// Example in Windows Forms

```
textBox1.PasswordChar = '*'; // This masks input with '*'
```

You can use any character (e.g., •, #, etc.) to mask the input.

### Alternative (UseSystemPasswordChar):

You can also use the system default password character which displays dot(.).

```
textBox1.UseSystemPasswordChar = true;
```

## 8. Differentiate Panel and GroupBox.

### Difference between Panel and GroupBox

| Feature        | Panel                                                               | GroupBox                                                                             |
|----------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Definition     | A simple container control used to group and manage other controls. | A container control used to logically group related controls with a visible caption. |
| Caption / Text | Cannot display a caption (no Text property).                        | Can display a caption using the Text property (e.g., "User Info").                   |

|                   |                                                                                     |                                                                               |
|-------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Border            | Has a border style (e.g., None, FixedSingle, Fixed3D).                              | Always shows a border with a title (caption).                                 |
| Use of Scrollbars | Supports AutoScroll property, so it can display scroll bars when content is larger. | Does not support scrollbars.                                                  |
| Controls Property | Can contain multiple controls; useful for layout management.                        | Can contain multiple controls; mainly for grouping related controls visually. |
| Common Usage      | For layout, scrolling, and organizing UI sections.                                  | For logically grouping related controls (e.g., radio buttons, checkboxes).    |

**Example:**

- **Panel:** Place many textboxes inside a scrollable form section.
- **GroupBox:** Place radio buttons inside a titled box called "**Choose Gender**".

**9. Write the purpose of AutoScroll and Controls properties of Panel.****a) AutoScroll**

- Purpose:
  - Determines whether the Panel automatically provides scroll bars when its contents are larger than the visible area.
  - When set to true, horizontal and/or vertical scroll bars will appear as needed.
- Example:
- `panel1.AutoScroll = true;`

If controls inside the panel extend beyond its boundary, scrollbars will appear automatically.

**b) Controls**

- Purpose:
  - Represents the collection of child controls contained within the Panel.
  - Using this property, you can add, remove, or access controls inside the panel.
- Example:
- `panel1.Controls.Add(button1);` // Add a button to the panel
- `panel1.Controls.Remove(textBox1);` // Remove a textbox from the panel

`var count = panel1.Controls.Count;` // Get number of controls in the panel

## 10. Differentiate RadioButton and CheckBox.

### Difference between RadioButton and CheckBox in C# Windows Forms:

| Feature          | RadioButton                                                                                 | CheckBox                                                                        |
|------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Selection type   | Allows the user to select only one option from a group (mutually exclusive).                | Allows the user to select multiple options independently.                       |
| Grouping         | When placed inside the same GroupBox/Panel, only one RadioButton can be selected at a time. | Each CheckBox works independently; multiple can be checked simultaneously.      |
| Appearance       | Small circular button with a dot inside when selected.                                      | Small square box with a tick (✓) mark when selected.                            |
| Default Behavior | Selecting one automatically deselects others in the same group.                             | Selecting one does not affect others.                                           |
| Use Case         | Best for single-choice questions, like selecting Gender (Male/Female/Other).                | Best for multi-choice options, like selecting Hobbies (Reading, Music, Sports). |

#### Example:

- RadioButton: *Choose Payment Method* → (Credit Card / Debit Card / UPI)
- CheckBox: *Select Languages Known* → (English, Hindi, French)

### Three or More Marks Questions

#### 1. List any 8 controls with their purpose. (4)

| Control       | Description                                                                                                                               |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Label         | Displays images or uneditable text.                                                                                                       |
| TextBox       | Enables the user to enter data via the keyboard. It can also be used to display editable or uneditable text.                              |
| Button        | Triggers an event when clicked with the mouse.                                                                                            |
| CheckBox      | Specifies an option that can be selected (checked) or unselected (not checked).                                                           |
| ComboBox      | Provides a drop-down list of items from which the user can make a selection either by clicking an item in the list or by typing in a box. |
| ListBox       | Provides a list of items from which the user can make a selection by clicking one or more items.                                          |
| Panel         | A container in which controls can be placed and organized.                                                                                |
| NumericUpDown | Enables the user to select from a range of numeric input values.                                                                          |

## 2. Explain AcceptButton, AutoScroll, CancelButton, FormBorderStyle and Font properties of Windows Form control. (5)

### a). AcceptButton

- Purpose: Specifies the button that is clicked when the user presses the Enter key.
- Usage: Typically set to the OK or Submit button on a form.
- Example:
- `this.AcceptButton = this.okButton;`

### b). AutoScroll

- Purpose: Enables scrollbars on the form if the form's content exceeds its visible area.
- Usage: Useful when displaying dynamic or large content.
- Example:
- `this.AutoScroll = true;`

### c). CancelButton

- Purpose: Specifies the button that is clicked when the user presses the Esc key.
- Usage: Typically set to the Cancel or Close button.
- Example:
- `this.CancelButton = this.cancelButton;`

### d). FormBorderStyle

- Purpose: Sets the appearance and behavior of the form's border.
- Common Values:
  - FixedSingle
  - Fixed3D
  - Sizable
  - None (for borderless forms)
- Example:
- `this.FormBorderStyle = FormBorderStyle.FixedSingle;`

### e). Font

- Purpose: Defines the font style, size, and family used by the form and its controls (if inherited).
- Usage: Applied to enhance UI readability and design.
- Example:

`this.Font = new Font("Arial", 10, FontStyle.Bold);`

### 3. Explain how delegates can be used in event handling. (4)

Delegates Event handlers are connected to a control's events via special objects called delegates.

A **delegate** is a **type-safe function pointer**. It can reference any method with a matching signature and return type.

```
public delegate void MyDelegate(string message);
```

#### Built-in Delegate Types

C# also provides built-in delegates like:

- Action – for methods with no return value.
- Func<T> – for methods with return values.
- EventHandler – a common delegate used for event handling:

```
public event EventHandler MyEvent;
```

**Event** An instance based on a delegate that notifies subscribers.

**Event Handler** A method that matches the delegate signature and is invoked when the event fires.

**+=** Used to subscribe to the event.

**-=** Used to unsubscribe.

### 4. Explain the following properties with respect to Control Class (6)

**a) Enabled b) ForeColor c) BackColor d) Text e) TabIndex f) TabStop**

#### a) Enabled

Purpose: Determines whether the control can respond to user interactions (like mouse clicks or keyboard input).

True: Control is active and can be used.

False: Control is disabled (appears grayed out and doesn't respond).

Example: `button1.Enabled = false;` → Button becomes inactive.

#### b) ForeColor

- Purpose: Defines the color of the control's text or foreground elements.
- Commonly affects labels, button text, textbox input, etc.
- Example: `label1.ForeColor = Color.Red;` → The text of the label turns red.

#### c) BackColor

- Purpose: Sets the background color of the control.

- Useful for making UI visually distinct.
- Example: `textBox1.BackColor = Color.LightYellow;` → Textbox background turns yellow.

**d) Text**

- Purpose: Represents the displayed text of the control.
- For controls like Label, Button, TextBox, Form, etc.
- Example: `button1.Text = "Submit";` → The button caption becomes "Submit".

**e) TabIndex**

- Purpose: Defines the order of focus when the user navigates controls with the Tab key.
- Smaller values get focus first.
- Example:
  - `textBox1.TabIndex = 0;`
  - `textBox2.TabIndex = 1;` → Pressing Tab moves from textBox1 to textBox2.

**f) TabStop**

- Purpose: Specifies whether the control can receive focus when pressing the Tab key.
- True: Control can be focused with Tab.
- False: Control is skipped when tabbing.
- Example: `checkBox1.TabStop = false;` → The checkbox will not be focused when pressing Tab.

**5. Explain about anchoring and docking a control. (5)**

In C# Windows Forms, anchoring and docking are two layout techniques used to control how controls (like buttons, textboxes, etc.) behave when the form is resized.

**a). Anchoring (2 Marks)**

- Definition: Anchoring ties a control to one or more edges of its parent container (like a Form or Panel).
- Purpose: Maintains the control's distance from the specified edges even when the form is resized.
- Example:  
If you anchor a Button to the Bottom and Right, it will stay in the bottom-right corner even if the form is resized.

`button1.Anchor = AnchorStyles.Bottom | AnchorStyles.Right;`



**b). Docking (2 Marks)**

- Definition: Docking attaches a control to an edge (or fills) of its parent container.
- Purpose: Automatically resizes and repositions the control to align with the specified edge.
- Example:  
Docking a TextBox to the Top will make it stick to the top edge and stretch horizontally.

```
textBox1.Dock = DockStyle.Top;
```

**Difference (1 Mark)**

| Aspect      | Anchoring                            | Docking                              |
|-------------|--------------------------------------|--------------------------------------|
| Behavior    | Maintains position relative to edges | Stretches to fill the specified edge |
| Flexibility | More control over size/position      | Good for auto layout                 |

**6. Explain Padding, Location, Size, Anchor, Dock properties of control layout. (5)****a). Padding (1 Mark)**

Definition: Padding defines the internal spacing inside a container control (e.g., Panel), between its border and its child controls.

Type: Padding (with properties: Left, Top, Right, Bottom)

Purpose: Adds space around the inner contents of a control.

Example:

```
panel1.Padding = new Padding(10); // 10px on all sides
```

**b). Location (1 Mark)**

Definition: Specifies the X and Y coordinates of the upper-left corner of the control relative to its parent container.

Type: Point

Purpose: Controls where the control appears inside its parent.

Example:

```
button1.Location = new Point(50, 100);
```

**c). Size (1 Mark)**

Definition: Specifies the width and height of the control.

Type: Size

Purpose: Determines how big the control is.

Example:

```
button1.Size = new Size(120, 40);
```

**d). Anchor (1 Mark)**

Definition: Binds the control to one or more sides of its parent to maintain relative position when resizing.

Type: AnchorStyles enum

Purpose: Makes the control responsive to form size changes.

Example:

```
textBox1.Anchor = AnchorStyles.Top | AnchorStyles.Right;
```

**e). Dock (1 Mark)**

Definition: Defines which edge of the parent container a control is docked to.

Type: DockStyle enum (Top, Bottom, Left, Right, Fill, None)

Purpose: Automatically resizes and aligns the control to the chosen edge.

Example:

```
panel1.Dock = DockStyle.Fill;
```

**7. Explain the following TextBox properties (5)****a) AcceptReturn****b) MultiLine****c) ReadOnly****d) ScrollBars****e) UseSystemPasswordChar****i. AcceptReturn**

**Type:** bool (True/False)

**Purpose:** Determines whether pressing the **ENTER (Return) key** creates a new line in the TextBox.

Works **only when Multiline = true**.

Example:

If AcceptReturn = true, pressing **Enter** moves the cursor to a new line.

If AcceptReturn = false, pressing **Enter** may trigger the form's default button (like an OK button).

**ii. Multiline**

**Type:** bool

**Purpose:** Specifies whether the TextBox can display **multiple lines of text**.

When set to true:

Text wraps or allows scrollbars.

You can press **Enter** (if AcceptReturn = true) to create new lines.

**iii. ReadOnly****Type:** bool**Purpose:** Makes the TextBox **non-editable** by the user while still showing its content.

Difference from Enabled = false:

ReadOnly = true → User cannot edit, but text looks normal.

Enabled = false → TextBox is grayed out and disabled.

**iv. ScrollBars****Type:** ScrollBars enumeration (None, Horizontal, Vertical, Both)**Purpose:** Determines which **scroll bars** (if any) appear in a **multiline** TextBox.

Works only when Multiline = true.

Example:

ScrollBars = Vertical → Vertical scrollbar appears for long text.

ScrollBars = Both → Both horizontal and vertical scrollbars are shown.

**v. UseSystemPasswordChar****Type:** bool**Purpose:** Masks characters typed in the TextBox with the system-defined password character (like • or \*) for **password entry fields**.

Example:

If set to true, typing “hello” will display “•••••”.

Often used for login/password fields.

**8. Explain the following properties of CheckBox (5)****a) Appearance b) Checked c) CheckState d) Text e) ThreeState****a) Appearance**

Type: Appearance (enum: Normal, Button)

Description:

This property changes how the CheckBox is visually rendered.

| Value  | Description                                                              |
|--------|--------------------------------------------------------------------------|
| Normal | Default look: a square box with a label.                                 |
| Button | Renders the checkbox like a toggle button that can be checked/unchecked. |

Example:

checkBox1.Appearance = Appearance.Button;

**b) Checked**

Type: bool

Description:

Determines whether the CheckBox is currently checked (true) or not (false).

Usage:

You can use it to programmatically check/uncheck a checkbox.

It also reflects the user's action at runtime.

Example:

```
bool isChecked = checkBox1.Checked;
```

```
checkBox1.Checked = true; // Sets it to checked
```

**c) CheckState**

Type: CheckState (enum: Unchecked, Checked, Indeterminate)

Description:

Gives more detailed state than just Checked. Especially useful when ThreeState is enabled.

| Value         | Description                                         |
|---------------|-----------------------------------------------------|
| Unchecked     | Not checked                                         |
| Checked       | Checked                                             |
| Indeterminate | Neither checked nor unchecked (e.g., unknown state) |

Example:

```
checkBox1.CheckState = CheckState.Indeterminate;
```

**d) Text**

Type: string

Description:

Sets or gets the text label displayed next to the checkbox.

Usage:

Useful to describe what the checkbox is for.

Example:

```
checkBox1.Text = "Accept Terms and Conditions";
```

**e) ThreeState**

Type: bool

Description:

Enables or disables a third state (Indeterminate) for the checkbox.

When set to true, the checkbox can have three states.

When set to false (default), it can only be Checked or Unchecked.

Example:

```
checkBox1.ThreeState = true;
```

```
checkBox1.CheckState = CheckState.Indeterminate;
```

### 9. Explain how to combine font styles using bitwise operators with examples. (5)

Combining font styles using bitwise operators is a common technique in programming to manage multiple options or settings compactly using a single integer value, especially in GUI frameworks or graphics libraries.

Font styles are often represented as flags, where each style is assigned a unique power of 2:

Font Style Binary Decimal

Bold 0001 1

Italic 0010 2

Underline 0100 4

Strikeout 1000 8

You can combine multiple styles using the bitwise OR (|) operator.

You can check for a specific style using the bitwise AND (&) operator.

#### Combine Styles with Bitwise OR

You can combine multiple styles by using the bitwise OR operator |:

```
FontStyle style = FontStyle.Bold | FontStyle.Italic;
```

This means the font will be **bold and italic** at the same time.

#### Combined style includes a specific style, use bitwise AND &:

```
if ((combinedStyle & FontStyle.Bold) == FontStyle.Bold)
```

```
{
```

```
    Console.WriteLine("Bold is applied");
```

```
}
```

The **bitwise NOT** operator (~) in C# flips all bits in an integer. When dealing with FontStyle enums, it can be used to *remove* certain styles from a combination by inverting bits and then applying bitwise AND.

#### // Remove Underline style

```
combinedStyle = combinedStyle & ~FontStyle.Underline;
```

## 10. Explain ToolTip control with its specific properties. (5)

The ToolTip control in C# Windows Forms is a component that provides short, helpful text messages when the user hovers the mouse pointer over a control (like a button, textbox, etc.).

It helps users understand the purpose or function of a control without taking extra space on the form.

### Properties of ToolTip Control

| Property            | Description                                                                                      | Example / Use                                          |
|---------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <b>ToolTipTitle</b> | Sets the <b>title text</b> displayed at the top of the tooltip window.                           | <code>toolTip1.ToolTipTitle = "Save Button";</code>    |
| <b>ToolTipIcon</b>  | Displays a small <b>icon</b> (Info, Warning, Error, or None) beside the tooltip text.            | <code>toolTip1.ToolTipIcon = ToolTipIcon.Info;</code>  |
| <b>IsBalloon</b>    | When set to true, the tooltip appears in a <b>balloon-style</b> popup.                           | <code>toolTip1.IsBalloon = true;</code>                |
| <b>AutoPopDelay</b> | The time (in milliseconds) that the tooltip remains visible after it appears.                    | <code>toolTip1.AutoPopDelay = 5000; (5 seconds)</code> |
| <b>InitialDelay</b> | The time delay (in milliseconds) before the tooltip appears when the user hovers over a control. | <code>toolTip1.InitialDelay = 1000; (1 second)</code>  |

## 11. Explain NumericUpDown control with its unique properties. (5)

### NumericUpDown Control

The NumericUpDown control in Windows Forms is a user interface element that allows users to select a numeric value by either typing it in or using up and down arrow buttons to increment or decrement the value. It is commonly used for input where only numeric values within a specific range are valid.

### Unique Properties of NumericUpDown

1. **Minimum**  
Specifies the lowest numeric value the control can have.
2. **Maximum**  
Specifies the highest numeric value the control can have.
3. **Value**  
Gets or sets the current numeric value of the control.
4. **Increment**  
Defines the amount by which the value increases or decreases when the up or down buttons are clicked.
5. **DecimalPlaces**  
Determines how many decimal places are displayed and allowed in the value.

Summary Example:

```
numericUpDown1.Minimum = 0;  
numericUpDown1.Maximum = 100;  
numericUpDown1.Value = 10;  
numericUpDown1.Increment = 5;  
numericUpDown1.DecimalPlaces = 2;
```

## 12. Explain the mouse events and mouse event arguments. (5)

### Mouse Events (in Windows Forms or similar environments):

Mouse events are triggered in response to **user interactions with the mouse**, such as clicking, moving, or releasing the mouse button.

Some common mouse events include:

| Event Name       | Description                                                     |
|------------------|-----------------------------------------------------------------|
| MouseClick       | Occurs when the control is clicked using any mouse button.      |
| MouseDown        | Triggered when a mouse button is <b>pressed down</b> .          |
| MouseUp          | Triggered when a mouse button is <b>released</b> .              |
| MouseMove        | Occurs when the mouse <b>moves over a control</b> .             |
| MouseEnter       | Triggered when the mouse <b>enters</b> the bounds of a control. |
| MouseLeave       | Triggered when the mouse <b>leaves</b> the bounds of a control. |
| MouseHover       | Occurs when the mouse hovers over a control for a short time.   |
| MouseDoubleClick | Triggered on a <b>double click</b> with the mouse.              |

### Mouse Event Arguments (MouseEventArgs):

When a mouse event is raised, it sends an object of type MouseEventArgs, which contains information about the mouse event.

### Important properties of MouseEventArgs:

#### Property Description

|         |                                                               |
|---------|---------------------------------------------------------------|
| Button  | Which mouse button was pressed (Left, Right, Middle).         |
| Clicks  | Number of clicks (e.g., 2 for double-click).                  |
| X and Y | The <b>coordinates</b> of the mouse pointer during the event. |
| Delta   | The amount the mouse wheel moved.                             |

## Property Description

Location A Point representing the (X, Y) position.

### 13. List the EventArgs class properties with their purpose (5)

#### EventArgs Class Properties and Their Purpose

| Property         | Purpose / Description                                                                                                  |
|------------------|------------------------------------------------------------------------------------------------------------------------|
| KeyCode          | Gets the keyboard key that was pressed. (Example: Keys.A, Keys.Enter)                                                  |
| KeyData          | Gets the key code combined with modifier keys (Ctrl, Alt, Shift). Useful to check for key combinations.                |
| KeyValue         | Returns the integer value representing the key that was pressed.                                                       |
| Modifiers        | Gets the modifier keys (Ctrl, Alt, Shift) that were pressed with the main key.                                         |
| Alt              | Returns true if the Alt key was pressed.                                                                               |
| Control          | Returns true if the Ctrl key was pressed.                                                                              |
| Shift            | Returns true if the Shift key was pressed.                                                                             |
| Handled          | Gets or sets a value indicating whether the event was handled. Setting it to true stops further processing of the key. |
| SuppressKeyPress | When set to true, it prevents the key character from being passed to the control. Useful for blocking input.           |

### 14. Explain the following properties of the ToolStripMenuItem (5)

a) CheckOnClick b) ShortcutKeyDisplayString c) ShortcutKeys d) ShowShortcutKeys e) Checked.

#### a) CheckOnClick

- Description:  
When set to true, this property allows the menu item to automatically toggle its checked state each time it is clicked.
- Effect:
  - If CheckOnClick = true, clicking the item will check it if it was unchecked, and uncheck it if it was checked.
  - If false, the check state must be changed manually in code.
- Example:
- menuItem.CheckOnClick = true;



**b) ShortcutKeyDisplayString**

- Description:  
This property allows you to manually set a custom text string that is displayed next to the menu item to indicate a shortcut key combination.
- Effect:
  - It does not assign the actual shortcut functionality — it only affects what text is displayed in the menu.
  - Useful when you want to show a shortcut that doesn't match an actual `ShortcutKeys` setting.
- Example:
- `menuItem.ShortcutKeyDisplayString = "Ctrl+Alt+M";`

**c) ShortcutKeys**

- Description:  
Defines the actual keyboard shortcut combination that activates the menu item.
- Effect:
  - When the user presses the assigned key combination, the menu item's click event is triggered.
  - You can assign combinations like `Ctrl + S`, `Alt + F4`, etc.
- Example:
- `menuItem.ShortcutKeys = Keys.Control | Keys.S;`

**d) ShowShortcutKeys**

- Description:  
Determines whether the shortcut key (set by `ShortcutKeys`) is displayed next to the menu item text in the menu.
- Effect:
  - If `true` → The shortcut key appears on the menu (e.g., "`Ctrl+S`").
  - If `false` → The shortcut is still active but not shown in the menu.
- Example:
- `menuItem.ShowShortcutKeys = true;`

**e) Checked**

- Description:  
Indicates whether the menu item is currently checked (has a tick mark next to it).
- Effect:
  - A visual check mark appears beside the item when `true`.

- Often used for toggle or state-type menu items (e.g., “Show Toolbar”).
- Example:
- `menuItem.Checked = true; // Displays a check mark next to the menu item`

### 15. Explain MonthCalendar control with any of its 4 unique properties. (5)

The **MonthCalendar** control in Windows Forms (WinForms) is used to **display a calendar interface** that allows users to **select one or more dates**.

It is commonly used in applications that require **date picking**, such as booking systems, forms, or scheduling tools.

#### Four Unique Properties of MonthCalendar:

| Property                  | Description                                                                             |
|---------------------------|-----------------------------------------------------------------------------------------|
| MaxSelectionCount         | Specifies the <b>maximum number of days</b> that can be selected at once. Default is 7. |
| SelectionStart            | Gets or sets the <b>starting date</b> of the selected range.                            |
| SelectionEnd              | Gets or sets the <b>ending date</b> of the selected range.                              |
| ShowToday                 | Determines whether the <b>today's date</b> is shown at the bottom of the calendar.      |
| FirstDayOfWeek<br>(bonus) | Sets which day appears as the <b>first day</b> of the week (e.g., Sunday, Monday).      |

### 16. Explain DateTimePicker control with any of its 4 unique properties. (5)

In C# Windows Forms, the DateTimePicker control is a user interface element that allows users to select a date, time, or both, from a calendar-style dropdown. It simplifies handling date and time input in applications.

The DateTimePicker control is found in the System.Windows.Forms namespace. It provides a graphical interface for selecting a date (and optionally time), ensuring input is in a valid DateTime format.

#### Four Unique Properties of DateTimePicker

| Property | Description                                                                                             | Example                                                          |
|----------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Format   | Determines how the date/time is displayed in the control. Common options: Short, Long, Time, or Custom. | <code>dateTimePicker.Format = DateTimePickerFormat.Short;</code> |
| Value    | Gets or sets the currently selected date and time.                                                      | <code>DateTime selectedDate = dateTimePicker.Value;</code>       |

|                   |                                                                                                           |                                                                                                              |
|-------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| MinDate / MaxDate | Defines the minimum and maximum date the user can select.                                                 | <code>dateTimePicker.MinDate = new DateTime(2000, 1, 1);<br/>dateTimePicker.MaxDate = DateTime.Today;</code> |
| ShowUpDown        | Displays a spin control (without a drop-down calendar) to adjust date/time values. Useful for time input. | <code>dateTimePicker.ShowUpDown = true;</code>                                                               |

### 17. Explain the following properties of LinkLabel control. (5)

a) **ActiveLinkColor** b) **LinkArea** c) **LinkBehavior** d) **LinkVisited** e) **UseMnemonic**

#### Properties of LinkLabel

| Property                  | Description                                                                                                                                                                                                     | Example                                                                                                                                           |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| a) <b>ActiveLinkColor</b> | Specifies the color of the link text when it is clicked (i.e., when it is active). It changes temporarily while the user presses the link.                                                                      | <code>linkLabel1.ActiveLinkColor = Color.Red;</code><br>The link turns red while being clicked.                                                   |
| b) <b>LinkArea</b>        | Determines which part of the text in the LinkLabel acts as a clickable link. It is defined using a LinkArea structure specifying start position and length of the link text.                                    | <code>linkLabel1.Text = "Click here to open website"; linkLabel1.LinkArea = new LinkArea(6, 4);</code><br>Only the word "here" will be clickable. |
| c) <b>LinkBehavior</b>    | Specifies the appearance and behavior of the link text — for example, whether it is always underlined or only when hovered. Values include: SystemDefault, AlwaysUnderline, HoverUnderline, and NeverUnderline. | <code>linkLabel1.LinkBehavior = LinkBehavior.HoverUnderline;</code><br>👉 The link underlines only when the mouse hovers over it.                  |
| d) <b>LinkVisited</b>     | Indicates whether the link has been visited (clicked) or not. When true, the link color                                                                                                                         | <code>linkLabel1.LinkVisited = true;</code><br>The link appears in the visited color (usually purple).                                            |

|                   |                                                                                                                                                                                |                                                                                                            |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
|                   | changes to the VisitedLinkColor.                                                                                                                                               |                                                                                                            |
| e)<br>UseMnemonic | Determines whether the control interprets an ampersand (&) in the text as a keyboard mnemonic (shortcut).<br>When true, the character following & becomes an Alt-key shortcut. | linkLabel1.UseMnemonic = true;<br>linkLabel1.Text = "Go to &Home";<br>Pressing Alt + H activates the link. |

### 18. Explain how to add, remove and clear items from a ListBox control with code snippets. (5)

#### Add Items to ListBox:

```
// Adding a single item
```

```
listBox1.Items.Add("Apple");
```

```
// Adding multiple items
```

```
listBox1.Items.AddRange(new string[] { "Harshith", "Joshua", "Samhitha" });
```

#### Remove Items from ListBox:

```
// Remove item by value
```

```
listBox1.Items.Remove("Samhitha");
```

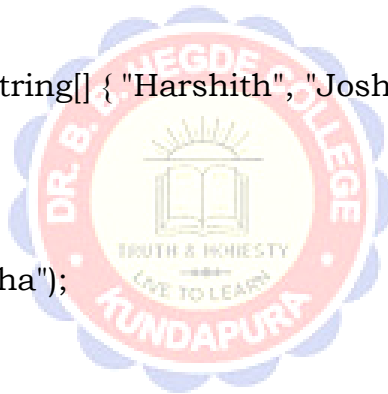
```
// Remove item by index
```

```
listBox1.Items.RemoveAt(0);
```

#### Clear All Items:

```
// Clears all items from the ListBox
```

```
listBox1.Items.Clear();
```



### 19. Explain any 5 unique properties of CheckedListBox. (5)

5 unique properties of the CheckedListBox control in Windows Forms (WinForms), commonly used in C#:

#### 1. CheckOnClick

- Description: When set to true, checking or unchecking an item happens with a single click.
- Default: false
- Usage: Useful for quick selection without needing to double-click or use the spacebar.

## 2. CheckedItems

- Description: Returns a collection (CheckedListBox.CheckedItemCollection) of the items that are currently checked.
- Type: CheckedListBox.CheckedItemCollection
- Usage: You can iterate through this to see which items the user selected.

```
foreach (var item in checkedListBox1.CheckedItems)
{
    MessageBox.Show(item.ToString());
}
```

## 3. CheckedIndices

- Description: Returns the indices (integers) of the currently checked items.
- Type: CheckedListBox.CheckedIndexCollection
- Usage: Helpful when you need to reference items by index rather than value.

## 4. ThreeDCheckBoxes

- Description: Determines whether the checkboxes use a 3D or flat style appearance.
- Type: bool
- Usage: Enables a more modern or customized UI look.

## 5. ItemCheck

- Description: An event, not a property, but crucial and unique to this control. It is triggered before the check state changes.
- Usage: Lets you intercept or cancel the check/uncheck action, or update UI accordingly.

## 20. Explain ComboBox control with at least 5 of its unique properties. (5)

5 Unique Properties of ComboBox

### 1. DropDownStyle

- Description: Specifies the style of the ComboBox.
- Values:
  - DropDown (editable + drop-down list)
  - DropDownList (non-editable, like a drop-down menu)
  - Simple (always visible list)
- Example:
- `comboBox1.DropDownStyle = ComboBoxStyle.DropDownList;`

## 2. Items

- Description: Gets the collection of items in the ComboBox.
- Type: ObjectCollection
- Usage: Add or remove items dynamically.

```
comboBox1.Items.Add("Apple");
```

```
comboBox1.Items.Add("Banana");
```

## 3. SelectedItem

- Description: Gets or sets the currently selected item (by value).
- Type: object
- Example:
- `var selected = comboBox1.SelectedItem;`

## 4. SelectedIndex

- Description: Gets or sets the index of the currently selected item.
- Type: int
- Usage: Use -1 to deselect.

```
comboBox1.SelectedIndex = 0; // selects the first item
```

## 5. AutoCompleteMode *(in combination with AutoCompleteSource)*

- Description: Provides automatic text completion.
- Values: None, Suggest, Append, SuggestAppend
- Must be used with AutoCompleteSource
- Example:
- `comboBox1.AutoCompleteMode = AutoCompleteMode.SuggestAppend;`

```
comboBox1.AutoCompleteSource = AutoCompleteSource.ListItems;
```

## 21. Explain MDI form with relevant properties. (5)

MDI Form (Multiple Document Interface) in Windows Forms

An MDI Form is a container form that can hold multiple child forms within a single parent window. It's commonly used in applications like text editors or graphic editors where users can work on multiple documents simultaneously (e.g., Microsoft Word 2003).

Key Concepts of MDI

- MDI Parent: The main window that contains other child windows.
- MDI Child: A form that opens within the parent, not as a separate window.

## Relevant Properties of MDI Forms

### i. IsMdiContainer

- Description: Indicates whether the form is an MDI parent.
- Type: bool
- Usage:
  - `this.IsMdiContainer = true; // Set in the parent form`

### ii. MdiParent

- Description: Sets or gets the parent form of an MDI child.
- Type: Form
- Usage:
  - `Form child = new Form();`
  - `child.MdiParent = this; // 'this' is the MDI parent`
  - `child.Show();`

### iii. MdiChildren

- Description: Returns an array of all MDI child forms currently open in the parent.
- Type: Form[]
- Usage:
  - `foreach (Form f in this.MdiChildren)`
  - {
  - `f.Close(); // Closes all open child forms`
  - }



### iv. ActiveMdiChild

- Description: Gets the currently active MDI child form.
- Type: Form
- Usage:
  - `Form activeChild = this.ActiveMdiChild;`

### v. LayoutMdi()

- Description: Arranges MDI child windows in different layouts (cascade, tile).
- Usage:
  - `this.LayoutMdi(MdiLayout.TileHorizontal);`

Options include:

- `MdiLayout.Cascade`

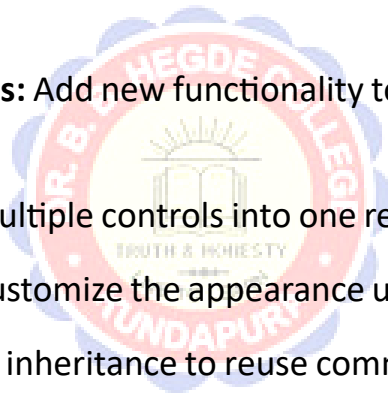
- MdiLayout.TileHorizontal
- MdiLayout.TileVertical
- MdiLayout.ArrangeIcons

### **Example of MDI Implementation**

```
// In MDI Parent Form  
this.IsMdiContainer = true;  
Form childForm = new Form();  
childForm.MdiParent = this;  
childForm.Text = "Child Window";  
childForm.Show();
```

### **22. Explain different techniques of custom control creation. (5)**

- **Deriving from Control class:** Create a control from scratch by overriding base methods.
- **Extending existing controls:** Add new functionality to existing controls (e.g., enhanced Button).
- **User controls:** Combine multiple controls into one reusable composite control.
- **Owner-drawn controls:** Customize the appearance using GDI+ drawing.
- **Inherited forms:** Use form inheritance to reuse common layouts across multiple forms.





**Unit – IV****TWO Marks Questions****1. List the steps to avoid concurrency errors.**

Concurrency errors in ASP.NET typically occur when multiple users or processes try to access or modify the same data simultaneously, leading to conflicts like data loss, overwrites, or inconsistent reads. To avoid these, you can use various strategies depending on your application architecture (e.g., Web Forms, MVC, Web API) and data access technology (e.g., ADO.NET, Entity Framework, Dapper).

**Steps to Avoid Concurrency Errors**

- I. **Use Concurrency Control**
  - a. Choose between:
    - i. **Optimistic Concurrency**: Allow multiple reads/writes, detect conflicts during save.
    - ii. **Pessimistic Concurrency**: Lock data during use to prevent others from changing it.
- II. **Implement Versioning (RowVersion / Timestamp)**
  - a. Add a version field to the database table.
  - b. Check this field before saving changes to ensure no other changes occurred.
- III. **Handle Concurrency Exceptions**
  - a. Catch and resolve `DbUpdateConcurrencyException` or similar errors when saving data.
  - b. Allow the user to retry, overwrite, or cancel.
- IV. **Use Transactions**
  - a. Group related operations into a transaction to maintain data consistency.
  - b. Roll back changes if something fails.
- V. **Avoid Long-Lived Data Contexts**
  - a. Use short-lived `DbContext` (or equivalent) to avoid stale data and long locks.
- VI. **Keep UI and Data in Sync**
  - a. Reload data or notify users if the data has changed before they submit changes.
- VII. **Minimize Simultaneous Access**
  - a. Break large operations into smaller units.
  - b. Reduce time spent holding locks or transactions open.
- VIII. **Log Changes and Track History**
  - a. Keep an audit trail to monitor and debug concurrency issues.
- IX. **Use Locks Where Necessary (Pessimistic Locking)**
  - a. Lock records for critical operations, but use sparingly to avoid performance issues.
- X. **Test for Concurrency Conflicts**

Simulate multiple users editing the same data to catch potential problems early.

## 2. Give the purpose of Connection and Command objects.

### Purpose of Connection and Command Objects in ADO.NET (used in ASP.NET)

These two objects are essential for database operations in ADO.NET, which is the core data access technology used in ASP.NET applications.

#### 1. Connection Object

Purpose:

- Establishes a link between your application and the database.
- Manages the opening, using, and closing of the database connection.

Example:

```
SqlConnection conn = new SqlConnection("YourConnectionStringHere");  
conn.Open();  
// Use the connection...  
conn.Close();
```

#### 2. Command Object

Purpose:

- Executes SQL queries or stored procedures against the database.

Key Functions:

- Run SQL statements (e.g., SELECT, INSERT, UPDATE, DELETE).
- Execute stored procedures.
- Pass parameters to queries.
- Retrieve results (via ExecuteReader, ExecuteScalar, ExecuteNonQuery).

Example class: SqlCommand

Example:

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Users", conn);  
SqlDataReader reader = cmd.ExecuteReader();
```

## 3. Give the purpose of DataReader and Data Adapter objects.

Purpose of DataReader and DataAdapter Objects in ADO.NET (used in ASP.NET)

Both DataReader and DataAdapter are used to retrieve data from a database, but they work in different ways based on how the data is accessed and manipulated.

#### 1. DataReader Object

Purpose:

- Used to read data from the database in a fast, forward-only, read-only manner.
- Ideal for quickly displaying data without needing to update it.

Example class: SqlDataReader

Example:

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Users", conn);
```

```
SqlDataReader reader = cmd.ExecuteReader();
```

## 2. DataAdapter Object

Purpose:

- Acts as a bridge between the database and a DataSet (or DataTable).
- Used for filling in-memory data (disconnected) and optionally updating the database.
- Works with commands: SelectCommand, InsertCommand, UpdateCommand, DeleteCommand.

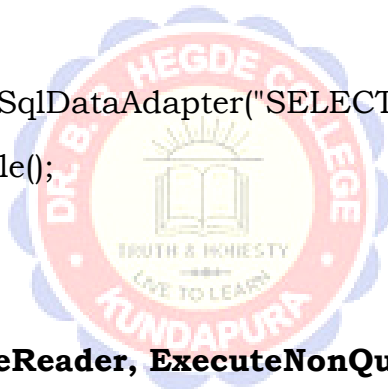
Example class: SqlDataAdapter

Example:

```
SqlDataAdapter adapter = new SqlDataAdapter("SELECT * FROM Users", conn);
```

```
DataTable table = new DataTable();
```

```
adapter.Fill(table);
```



## 3. Write the purpose of ExecuteReader, ExecuteNonQuery methods.

Purpose of ExecuteReader and ExecuteNonQuery Methods in ADO.NET

These methods belong to the Command object (like SqlCommand) and are used to execute different types of SQL queries against a database.

### 1. ExecuteReader()

Purpose:

- Executes a SQL SELECT statement and returns data from the database.
- Returns a DataReader object (e.g., SqlDataReader) to read rows forward-only and read-only.

Use When:

- You want to retrieve data (e.g., list of users, product info).
- Query returns multiple rows and columns.

Example:

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Users", conn);
```

```
SqlDataReader reader = cmd.ExecuteReader();
```

## 2. ExecuteNonQuery()

Purpose:

- Executes a SQL statement that does not return data.
- Commonly used for INSERT, UPDATE, DELETE, or DDL commands like CREATE TABLE.

Returns:

- The number of rows affected by the command.

Use When:

- You want to change data in the database but don't need to get any data back.

Example:

```
SqlCommand cmd = new SqlCommand("DELETE FROM Users WHERE Id = 5", conn);  
int rowsAffected = cmd.ExecuteNonQuery();  
Console.WriteLine($"{rowsAffected} row(s) deleted.");
```

## 5. Write a note on using DataSource window.

The DataSource window in C# ASP.NET (especially in Visual Studio) is a visual tool that simplifies the process of connecting to data sources like databases, web services, or business objects.

It is mainly used in ASP.NET Web Forms or Windows Forms projects to design data-driven applications more easily.

Purpose of the DataSource Window:

- Helps developers bind UI controls (like GridView, DropDownList, etc.) to data sources without writing much code.
- Provides a graphical way to manage database connections and generate data-bound controls.

## 6. Write the purpose of ComboBox properties that help data binding.

In C# ASP.NET a **ComboBox** is a control that allows users to select from a list of options. When working with **data binding**, certain properties of the ComboBox help link it to a data source efficiently.

### Purpose of ComboBox Properties that Help Data Binding

The key properties used for data binding in a ComboBox, along with their purposes:

#### 1. DataSource

Connects the ComboBox to the source of data (e.g., a database table, list, collection, or DataTable).

- **Example:**
- `comboBox.DataSource = myDataTable;`

## 2. DataTextField

Specifies the field from the data source that will be displayed as the text of each item in the ComboBox.

- **Example:**
- `comboBox.DataTextField = "ProductName";`

## 3. DataValueField

Specifies the field from the data source that will be used as the **value** of each item (usually used when submitting forms).

- **Example:**
- `comboBox.DataValueField = "ProductID";`

## 4. SelectedValue

Sets or gets the value of the currently selected item, typically used to pre-select an item during page load or retrieve the user's selection.

- **Example:**
- `comboBox.SelectedValue = "102";`

## 5. SelectedItem / SelectedIndex

SelectedItem returns the selected item object, and SelectedIndex gets or sets the index of the selected item. Useful for logic after user interaction.

- **Example:**
- `var selected = comboBox.SelectedItem;`
- `int index = comboBox.SelectedIndex;`

## 6. AppendDataBoundItems (for Web Forms)

Allows static items already in the ComboBox (e.g., "Select an option") to remain when data binding occurs.

- **Example:**
- `<asp:DropDownList ID="ddlExample" AppendDataBoundItems="True" ... >`
- `<asp:ListItem Text="--Select--" Value="" />`
- `</asp:DropDownList>`

## 7. Write the purpose of any 4 methods of BindingSource Class.

The BindingSource class in ASP.NET (more commonly in Windows Forms rather than ASP.NET Web Forms or MVC) is used to simplify data binding between UI controls and data sources like lists, databases, or objects.

### a. Add(Object value)

- Purpose: Adds a new item to the data source list.

```
bindingSource.Add(new Customer { Name = "Annappa", Age = 30 });
```

#### **b. Remove(Object value)**

- Purpose: Removes the specified item from the data source.

```
bindingSource.Remove(customerToDelete);
```

#### **c. MoveNext()**

- Purpose: Moves the current position in the list to the next item.

```
bindingSource.MoveNext();
```

#### **d. ResetBindings(Boolean metadataChanged)**

- Purpose: Forces the control(s) bound to the data source to refresh their displayed data.

```
bindingSource.ResetBindings(false); // Use true if the schema/structure changed
```

### **8. Write the purpose of the open() and dispose() methods of SqlConnection Class.**

#### Methods of SqlConnection Class

| Method    | Purpose / Description                                                                                                                                                                                                                                |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Open()    | Used to open a connection to a SQL Server database. It establishes the physical link between the application and the database using the connection string provided.<br>Example: <code>sqlConnection.Open();</code>                                   |
| Dispose() | Used to release all resources (like memory and network connections) used by the SqlConnection object. It ensures proper cleanup after the connection is no longer needed, preventing memory leaks.<br>Example: <code>sqlConnection.Dispose();</code> |

### **9. Write the purpose of Read() and NextResult() methods of SqlDataReader class.**

#### Methods of SqlDataReader Class

| Method       | Purpose / Description                                                                                                                                                                                                        |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Read()       | Moves the SqlDataReader to the next record (row) in the result set. It returns true if there are more rows to read, otherwise false.<br>Purpose: Used to loop through records one by one.                                    |
| NextResult() | Moves the SqlDataReader to the next result set when a query returns multiple result sets (e.g., from multiple SELECT statements in one command).<br>Purpose: Allows reading results from multiple queries executed together. |

## 10. What is ASP .NET?

ASP.NET (Active Server Pages .NET) is a web development framework created by Microsoft for building dynamic web applications, websites, and web services.

It is part of the .NET platform, and it enables developers to create server-side web applications using programming languages like C# or VB.NET.

## 11. Write the purpose of TextMode property of Web TextBox Control.

In ASP.NET, the TextMode property of a TextBox control specifies the type of text box that will be rendered on the web page and how the user can input data into it.

Syntax:

```
<asp:TextBox ID="TextBox1" runat="server" TextMode="SingleLine"></asp:TextBox>
```

### Possible Values of TextMode:

| Value      | Description                                                                               |
|------------|-------------------------------------------------------------------------------------------|
| SingleLine | Default. Allows the user to enter a single line of text (e.g., name, email).              |
| MultiLine  | Allows multiple lines of text; the control renders as a <textarea> in HTML.               |
| Password   | Masks the input characters for password entry (renders as <input type="password">).       |
| Email      | Accepts email addresses and can trigger email-specific keyboards on mobile devices.       |
| Number     | Restricts input to numeric values.                                                        |
| Date       | Allows users to select or enter a date using a date picker (if supported by the browser). |
| Time       | Allows time input using a time picker.                                                    |
| Url        | Accepts URL input.                                                                        |
| Phone      | Accepts telephone numbers and opens numeric keypads on mobile devices.                    |

## 12. Write the general form of an ASP .NET web control.

In ASP.NET, every Web Server Control (such as a TextBox, Button, Label, etc.) has a general form (syntax) that defines how it is declared within an ASPX page.

### General Form of an ASP.NET Web Control

```
<asp:ControlName
```

```
    ID="ControlID"
```

```

    runat="server"

    Property1="Value1"

    Property2="Value2"

    ... >

</asp:ControlName>

```

#### Explanation of Each Part:

| Part                      | Description                                                                                                                               |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <asp:                     | Indicates that it is an ASP.NET server control.                                                                                           |
| ControlName               | Specifies the type of control (e.g., TextBox, Button, Label).                                                                             |
| ID                        | A unique identifier for the control, used to access it in the server-side code (C#/VB).                                                   |
| runat="server"            | Makes the control a server-side control, so it can be processed on the server. <i>(Mandatory for ASP.NET controls)</i>                    |
| Property1, Property2, ... | Define the control's attributes or properties (such as Text, Width, CssClass, etc.).                                                      |
| </asp:ControlName>        | Closing tag for the control (for controls that have opening and closing tags). Some controls, like <asp:Label>, can also be self-closing. |

### 13. What is an Image control? Write any two of its unique properties.

The Image control in ASP.NET is a server control used to display an image on a web page. It allows you to dynamically set or change the image source and other related properties from the server-side code (C# or VB).

Example (C#):

```
Image1.ImageUrl = "~/Images/banner.jpg";
```

```
Image1.AlternateText = "Website Banner";
```

#### Two Unique Properties of Image Control

| Property      | Description                                                                                                                                                                |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ImageUrl      | Specifies the path or URL of the image to display. It can be a relative or absolute URL. Example: ImageUrl="~/Images/pic1.jpg".                                            |
| AlternateText | Specifies the text that will be displayed if the image cannot be loaded. It also helps with accessibility (screen readers). Example: AlternateText="User Profile Picture". |



#### 14. What is a Hyperlink control? List its unique properties.

The HyperLink control in ASP.NET is a server-side control used to create a clickable link that navigates the user to another web page, website, email address, or file.

##### Unique Properties of the HyperLink Control

| Property    | Description                                                                                                    |
|-------------|----------------------------------------------------------------------------------------------------------------|
| NavigateUrl | Specifies the destination URL the hyperlink points to. Example: NavigateUrl="https://www.microsoft.com".       |
| Text        | The visible text displayed for the hyperlink. Example: Text="Click Here".                                      |
| Target      | Specifies how the linked page is opened:<br>_blank → New window/tab,<br>_self → Same window,<br>_parent, _top. |
| ImageUrl    | Allows you to use an image instead of text as the hyperlink content. Example: ImageUrl="~/Images/home.png".    |
| ToolTip     | Displays a tooltip when the mouse hovers over the hyperlink.                                                   |

#### 15. Write a note on RequiredFieldValidator control.

- The RequiredFieldValidator is a validation control in **ASP.NET** that ensures a user **does not skip a mandatory field**.
- It triggers an error message if the associated input control is left **blank**.
- It is often used with TextBox, DropDownList, etc.
- Key property: ControlToValidate, ErrorMessage.

### Three or More Marks Questions

#### 1. Describe basic ADO .NET objects. (5)

Basic ADO.NET Objects – Description and Purpose

ADO.NET (ActiveX Data Objects for .NET) is a part of the .NET Framework used to interact with databases. It provides a set of classes that allow you to connect to a database, execute commands, and manage data.

Below are the basic ADO.NET objects that are commonly used:

##### 1. Connection

Purpose:

- Establishes a connection between the application and the database.

Example Class:

- SqlConnection (for SQL Server)

Example:

```
SqlConnection conn = new SqlConnection("your_connection_string");  
conn.Open();
```

## 2. Command

Purpose:

- Executes SQL queries or stored procedures against the database.

Example Class:

- SqlCommand

Example:

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Users", conn);
```

## 3. DataReader

Purpose:

- Reads data from the database in a fast, forward-only, read-only manner.

Example Class:

- SqlDataReader

Example:

```
SqlDataReader reader = cmd.ExecuteReader();  
while (reader.Read())  
{  
    Console.WriteLine(reader["Name"].ToString());  
}
```



## 4. DataAdapter

Purpose:

- Acts as a bridge between the database and a DataSet.
- Used to fill data and optionally update it back to the database.

Example Class:

- SqlDataAdapter

Example:

```
SqlDataAdapter adapter = new SqlDataAdapter("SELECT * FROM Users", conn);  
DataTable table = new DataTable();
```

```
adapter.Fill(table);
```

## 5. DataSet

Purpose:

- A disconnected, in-memory representation of data.
- Can hold multiple tables and relationships.

Example:

```
DataSet ds = new DataSet();  
adapter.Fill(ds, "Users");
```

## 6. DataTable

Purpose:

- Represents a single table of in-memory data.
- Often used with DataSet or directly with DataAdapter.

Example:

```
DataTable table = new DataTable();  
adapter.Fill(table);
```

## 7. DataView

Purpose:

- Provides a custom view (like sorting, filtering) of a DataTable.

Example:

```
DataView view = new DataView(table);  
view.RowFilter = "Age > 30";
```

## 2. Describe how we can use ADO .NET without using datasets with code. (5)

ADO.NET can be used without DataSets — this is commonly done using a DataReader for connected access or executing commands directly for data manipulation.

This approach is lightweight, faster, and ideal for simple or read-only operations.

When to Use ADO.NET Without DataSets?

- You want fast, forward-only reading of data (e.g., display list of users).
- You don't need to manipulate data offline.
- You want more control over database operations.

Code Example: ADO.NET Without DataSet

This example:

- Connects to a database
- Executes a SELECT query
- Uses SqlDataReader to read and display data  
(No DataSet or DataTable is used.)

**C# Code:**

```
using System;
```

```
using System.Data.SqlClient;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        // 1. Define connection string
```

```
        string connectionString =
```

```
"Server=your_server_name;Database=your_db_name;Trusted_Connection=True;";
```

```
        // 2. Create SQL query
```

```
        string query = "SELECT Id, Name, Email FROM Users";
```

```
        // 3. Use SqlConnection and SqlCommand
```

```
        using (SqlConnection conn = new SqlConnection(connectionString))
```

```
        {
```

```
            SqlCommand cmd = new SqlCommand(query, conn);
```

```
            try
```

```
            {
```

```
                conn.Open(); // Open connection
```

```
                // 4. Execute reader (no DataSet used)
```

```
                SqlDataReader reader = cmd.ExecuteReader();
```

```
                // 5. Read data
```

```
                while (reader.Read())
```

```
                {
```

```
                    int id = (int)reader["Id"];
```

```
                    string name = reader["Name"].ToString();
```

```
        string email = reader["Email"].ToString();  
        Console.WriteLine($"ID: {id}, Name: {name}, Email: {email}");  
    }  
    reader.Close(); // Always close reader  
}  
catch (Exception ex)  
{  
    Console.WriteLine("Error: " + ex.Message);  
} } }
```

### Explanation of Key Parts:

| Step | Component            | Purpose                                             |
|------|----------------------|-----------------------------------------------------|
| 1    | SqlConnection        | Connects to the database                            |
| 2    | SqlCommand           | Holds the SQL query                                 |
| 3    | ExecuteReader()      | Executes the command and returns a SqlDataReader    |
| 4    | SqlDataReader.Read() | Reads rows one by one                               |
| 5    | No DataSet used      | Data is processed immediately, not stored in memory |

### 3. Explain what happens when 2 users try to update same data row? (5)

When two users try to update the **same row of data** in a database at the same time, the following can happen depending on the system's **concurrency control strategy**:

#### i. Last Update Wins (No Concurrency Control)

- The last user who saves the changes **overwrites the previous one's update**.
- **Risk:** Data loss, as earlier changes are lost without warning.

#### ii. Optimistic Concurrency Control

- Each user works with a **local copy** of the data.
- Before saving, the system checks if the data has changed on the server.
- If the data was modified by someone else:
  - **Update is rejected**, and the user is prompted to **reload or merge** changes.
- Common in **disconnected environments** like web apps or ADO.NET.

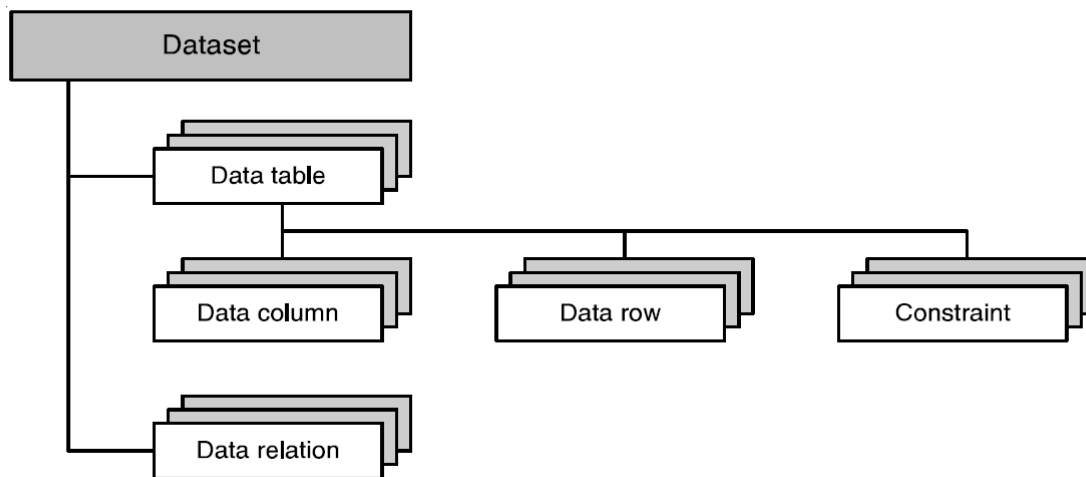
### iii. Pessimistic Concurrency Control

- When one user accesses the row, it is **locked** for others.
- Others must **wait** until the first user finishes.
- Prevents data conflicts but can reduce performance and cause **deadlocks**.

### 4. Explain the basic dataset object hierarchy with diagram.(5)

A DataSet is an **in-memory representation of data** that can hold **multiple tables, relations, and constraints**, commonly used in disconnected data access using ADO.NET.

#### The basic dataset object hierarchy



#### Description

- A dataset object consists of a hierarchy of one or more data table and *data relation* objects.
- A data table object consists of one or more *data column* objects and one or more *data row* objects. The data column objects define the data in each column of the table, including its name, data type, and so on, and the data row objects contain the data for each row in the table.
- A data table can also contain one or more *constraint* objects that are used to maintain the integrity of the data in the table. A unique key constraint ensures that the values in a column, such as the primary key column, are unique. And a foreign key constraint determines how the rows in one table are affected when corresponding rows in a related table are updated or deleted.
- The data relation objects define how the tables in the dataset are related. They are used to manage constraints and to simplify the navigation between related tables.
- All of the objects in a dataset are stored in collections. For example, the data table objects are stored in a data table collection, and the data row objects are stored in a data row collection. You can refer to these collections through the properties of the containing objects.

**5. Explain how to save connection string in app.config file. (4)****a. Add the connection string section (1 mark)**

In the App.config file, include a <connectionStrings> section inside the <configuration> element.

Example:

```
<configuration>

  <connectionStrings>

    <add name="MyDBConnection"

      connectionString="Data Source=SERVER_NAME;Initial
Catalog=DatabaseName;Integrated Security=True"

      providerName="System.Data.SqlClient" />

  </connectionStrings>

</configuration>
```

**b. Define key attributes (1 mark)**

- name – gives the connection a unique identifier (e.g., "MyDBConnection").
- connectionString – stores the database connection details.
- providerName – specifies the data provider (e.g., "System.Data.SqlClient" for SQL Server).

**c. Access the connection string in code (1 mark)**

In C#, retrieve it using the ConfigurationManager class:

```
using System.Configuration;

string connStr =
ConfigurationManager.ConnectionStrings["MyDBConnection"].ConnectionString;
```

**d. Explain the purpose (1 mark)**

Storing the connection string in App.config keeps database details separate from code, making the application easier to maintain, secure, and configurable without recompiling.

**6. Explain the steps to create connection to data base. (4)**

Here are the typical steps to create a connection to a database using **ADO.NET** in C# (e.g., for SQL Server):

**a. Import Required Namespace:**

```
using System.Data.SqlClient;
```

**b. Define Connection String:**

This string contains information about the database server, database name, and credentials.

```
string connectionString = "Data Source=SERVER_NAME;Initial  
Catalog=DATABASE_NAME;Integrated Security=True;"
```

**c. Create Connection Object:**

```
SqlConnection conn = new SqlConnection(connectionString);
```

**d. Open the Connection:**

```
conn.Open();
```

**e. Perform Database Operations:**

You can use a SqlCommand to execute SQL queries.

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Students", conn);
```

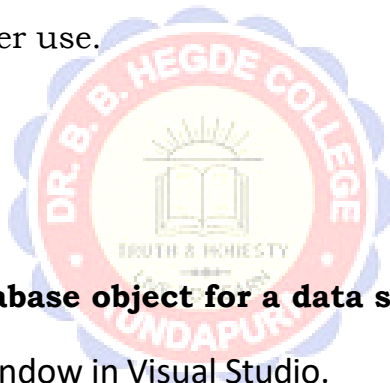
```
SqlDataReader reader = cmd.ExecuteReader();
```

**f. Close the Connection:**

Always close the connection after use.

```
reader.Close();
```

```
conn.Close();
```

**7. Explain how to choose database object for a data source? (4)**

- Open the **Data Sources** window in Visual Studio.
- Select **Add New Data Source** → choose **Database** → click **Next**.
- Configure the connection string and select database objects like tables or views.
- The selected objects are added to your project's data model for binding.

**8. Explain the controls and objects created when you drag a data source to a form. (5)**

- When you drag a data source onto a form, controls like **TextBox**, **Label**, or **DataGridView** are created automatically.
- A **BindingSource** object is added to manage data binding.
- A **TableAdapter** is created for communication with the database.
- A **DataSet** object is generated to hold the data.
- All these work together to simplify data display and updates in the form.



**9.Explain how to format the data displayed in TextBox. (5)**

Data displayed in a TextBox can be formatted to control how numbers, dates, or strings appear to the user. This is typically done by applying format strings, using data binding with formatting, or through string formatting methods like ToString().

**Methods to Format Data****Using ToString() with Format Specifiers**

Format the value before assigning it to the TextBox.

```
decimal price = 1234.567m;
```

```
textBox1.Text = price.ToString("C2"); // Displays as currency: $1,234.57
```

**Using String Interpolation or String.Format()**

```
DateTime today = DateTime.Now;
```

```
textBox1.Text = string.Format("{0:dd/MM/yyyy}", today); // Displays as 03/11/2025
```

**Using Data Binding with Formatting**

When a TextBox is bound to a data source, the FormatString property (through Binding) can be used:

```
textBox1.DataBindings.Add("Text", product, "Price", true,  
DataSourceUpdateMode.OnPropertyChanged, 0, "C2");
```

This binds and formats the Price field as currency.

**10.Explain how to bind a ComboBox to a data source. (5)**

A **ComboBox** in C#.NET can be **data bound** to display data from a source such as a **DataTable**, **List**, or **BindingSource**.

This allows items in the ComboBox to be filled automatically from the data source instead of adding them manually.

**Steps to Bind a ComboBox**

Create or fetch the data source (e.g., DataTable, List).

Set the DataSource property of the ComboBox.

Set the DisplayMember (field to show).

Set the ValueMember (field to store as value).

**Example Code**

```
using System;
```

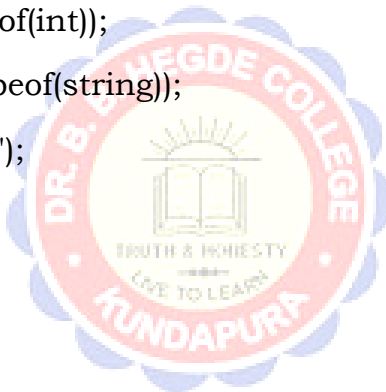
using System.Data;

using System.Windows.Forms;

```
public class ComboBoxBindingDemo : Form
{
    private ComboBox comboBox1;
    private DataTable dt;
    public ComboBoxBindingDemo()
    {
        comboBox1 = new ComboBox { Dock = DockStyle.Top };
        Controls.Add(comboBox1);
        // Create DataTable
        dt = new DataTable();
        dt.Columns.Add("ID", typeof(int));
        dt.Columns.Add("City", typeof(string));
        dt.Rows.Add(1, "New York");
        dt.Rows.Add(2, "London");
        dt.Rows.Add(3, "Paris");

        // Bind ComboBox to DataTable
        comboBox1.DataSource = dt;
        comboBox1.DisplayMember = "City"; // shown to user
        comboBox1.ValueMember = "ID";    // underlying value
    }

    [STAThread]
    public static void Main()
    {
        Application.Run(new ComboBoxBindingDemo());
    }
}
```



**11. Describe the usage of BindingSource class with code statements. (5)**

The BindingSource class in C#.NET is used to simplify data binding between UI controls (like DataGridView, TextBox, etc.) and a data source (like DataTable, List, or database).

It manages data navigation, updates, sorting, and filtering easily without writing complex code.

**Main Uses**

1. Acts as a connector between data source and UI controls.
2. Provides data navigation (MoveNext(), MovePrevious()).
3. Supports filtering and sorting of data.
4. Automatically updates controls when data changes.

**Example Code**

```
using System;
```

```
using System.Data;
```

```
using System.Windows.Forms;
```

```
public class BindingSourceDemo : Form
{
    private DataGridView dgv;
    private BindingSource bs;
    private DataTable dt;

    public BindingSourceDemo()
    {
        // Create DataGridView
        dgv = new DataGridView { Dock = DockStyle.Fill };
        Controls.Add(dgv);

        // Create a DataTable
        dt = new DataTable();
        dt.Columns.Add("ID", typeof(int));
        dt.Columns.Add("Name", typeof(string));
        dt.Rows.Add(1, "John");
        dt.Rows.Add(2, "Mary");
    }
}
```



```
dt.Rows.Add(3, "David");
```

```
// Create and set up BindingSource
```

```
bs = new BindingSource();
```

```
bs.DataSource = dt;
```

```
// Bind DataGridView to BindingSource
```

```
dgv.DataSource = bs;
```

```
}
```

```
[STAThread]
```

```
public static void Main()
```

```
{
```

```
Application.Run(new BindingSourceDemo());
```

```
}
```

```
}
```

### Explanation

- A DataTable is created with sample data.
- A BindingSource object is created and linked to the DataTable.
- The DataGridView control is then bound to the BindingSource.
- Any changes to data in the table or UI are automatically synchronized.

### 12. Explain how to work with DataGridView control. (5)

In **C# ASP.NET Web Forms**, you typically use the **GridView** control (not DataGridView, which is for **Windows Forms**) to display tabular data.

#### Bind Data to GridView

You can bind GridView to a data source like a DataTable, List, or SqlDataReader.

#### Example:

```
<asp:GridView ID="GridView1" runat="server"  
AutoGenerateColumns="true"></asp:GridView>
```

```
protected void Page_Load(object sender, EventArgs e)
```

```
{
```

```
if (!IsPostBack)
```

```
{  
    string connStr =  
    ConfigurationManager.ConnectionStrings["conStr"].ConnectionString;  
    using (SqlConnection con = new SqlConnection(connStr))  
    {  
        SqlDataAdapter da = new SqlDataAdapter("SELECT * FROM Students", con);  
        DataTable dt = new DataTable();  
        da.Fill(dt);  
        GridView1.DataSource = dt;  
        GridView1.DataBind();  
    } } }
```

### Enable Paging and Sorting

Allow users to navigate and sort through large datasets.

#### Example:

```
<asp:GridView ID="GridView1" runat="server"  
    AllowPaging="true"  
    PageSize="10"  
    AllowSorting="true"  
    OnPageIndexChanging="GridView1_PageIndexChanging">  
</asp:GridView>
```

#### Code-behind:

```
protected void GridView1_PageIndexChanging(object sender, GridViewPageEventArgs e)  
{  
    GridView1.PageIndex = e.NewPageIndex;  
    BindGrid(); // Rebind your data  
}
```

### Edit and Update Data

Enable editing by setting `AutoGenerateEditButton="true"` and handling `RowEditing`, `RowUpdating`, and `RowCancelingEdit` events.

#### Example:

```
<asp:GridView ID="GridView1" runat="server"  
    AutoGenerateEditButton="true"
```

```
OnRowEditing="GridView1_RowEditing"
```

```
OnRowUpdating="GridView1_RowUpdating"
```

```
OnRowCancelingEdit="GridView1_RowCancelingEdit">
```

```
</asp:GridView>
```

**Code-behind (simplified):**

```
protected void GridView1_RowEditing(object sender, GridViewEditEventArgs e)
```

```
{
```

```
    GridView1.EditIndex = e.NewEditIndex;
```

```
    BindGrid();
```

```
}
```

```
protected void GridView1_RowCancelingEdit(object sender,  
GridViewCancelEventArgs e)
```

```
{
```

```
    GridView1.EditIndex = -1;
```

```
    BindGrid();
```

```
}
```

```
protected void GridView1_RowUpdating(object sender, GridViewUpdateEventArgs e)
```

```
{
```

```
    int id = Convert.ToInt32(GridView1.DataKeys[e.RowIndex].Value);
```

```
    string name = ((TextBox)GridView1.Rows[e.RowIndex].Cells[1].Controls[0]).Text;
```

```
    string connStr =onfigurationManager.ConnectionStrings["conStr"].ConnectionString;
```

```
    using (SqlConnection con = new SqlConnection(connStr))
```

```
    {
```

```
        SqlCommand cmd = new SqlCommand("UPDATE Students SET Name=@Name  
WHERE Id=@Id", con);
```

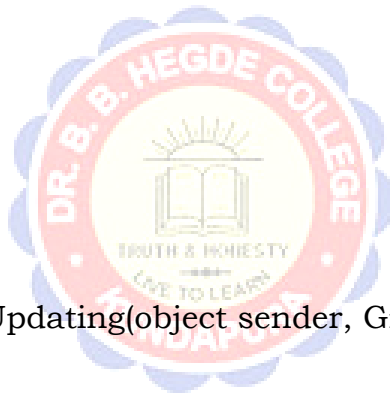
```
        cmd.Parameters.AddWithValue("@Name", name);
```

```
        cmd.Parameters.AddWithValue("@Id", id);
```

```
        con.Open();
```

```
        cmd.ExecuteNonQuery();
```

```
    }
```



```
GridView1.EditIndex = -1;  
BindGrid();  
}
```

### Delete Records

Enable delete functionality by setting `AutoGenerateDeleteButton="true"` and handling `RowDeleting` event.

```
<asp:GridView ID="GridView1" runat="server"  
    AutoGenerateDeleteButton="true"  
    OnRowDeleting="GridView1_RowDeleting">  
</asp:GridView>
```

### Code-behind:

```
protected void GridView1_RowDeleting(object sender, GridViewDeleteEventArgs e)  
{  
    int id = Convert.ToInt32(GridView1.DataKeys[e.RowIndex].Value);  
  
    string connStr =  
        ConfigurationManager.ConnectionStrings["conStr"].ConnectionString;  
    using (SqlConnection con = new SqlConnection(connStr))  
    {  
        SqlCommand cmd = new SqlCommand("DELETE FROM Students WHERE  
Id=@Id", con);  
        cmd.Parameters.AddWithValue("@Id", id);  
        con.Open();  
        cmd.ExecuteNonQuery();  
    }  
    BindGrid();  
}
```

### Customize Columns and Appearance

You can manually define columns, apply styles, and use templates.

### Example:

```
<asp:GridView ID="GridView1" runat="server" AutoGenerateColumns="false">
```

<Columns>

<BoundField DataField="Id" HeaderText="Student ID" />

<BoundField DataField="Name" HeaderText="Full Name" />

<TemplateField HeaderText="Custom">

<ItemTemplate>

Hello, <%# Eval("Name") %>

</ItemTemplate>

</TemplateField>

</Columns>

</asp:GridView>

### 13. Explain common properties of a column. (5)

In C#.NET, columns are used in data structures like DataGridview, DataTable, or database-bound controls to represent fields of data.

Each column has several common properties that control its name, data type, appearance, and behavior.

#### Common Properties of a Column

| Property         | Description                                                                                                |
|------------------|------------------------------------------------------------------------------------------------------------|
| Name             | Specifies the internal name of the column used in code.<br>Example: column.Name = "colID";                 |
| HeaderText       | Text displayed in the column header of a DataGridview.<br>Example: column.HeaderText = "Student ID";       |
| DataPropertyName | Binds the column to a specific field in the data source. Example:<br>column.DataPropertyName = "ID";       |
| Width            | Sets the width of the column in pixels. Example: column.Width = 100;                                       |
| ReadOnly         | Determines whether the user can edit the column's values.<br>Example: column.ReadOnly = true;              |
| DefaultCellStyle | Controls the appearance (font, color, format) of cells. Example:<br>column.DefaultCellStyle.Format = "C2"; |

#### Example Code

```
DataGridViewTextBoxColumn column = new DataGridViewTextBoxColumn();
column.Name = "colPrice";
column.HeaderText = "Product Price";
```



```
column.DataPropertyName = "Price";  
column.Width = 120;  
column.DefaultCellStyle.Format = "C2";  
dataGridView1.Columns.Add(column);
```

**Explanation**

- The above code creates a column for a DataGridView.
- Each property customizes how the column displays, binds, and behaves in the grid.

**14. Explain how to add an unbound column to a DataGridView control. (5)**

An unbound column is a column in a DataGridView that is not connected to any data source (like a database or DataTable).

It is used to display custom data, such as calculated values, serial numbers, or buttons, that are not part of the bound data source.

**Steps to Add an Unbound Column**

- Create a column object (e.g., DataGridViewTextBoxColumn, DataGridViewButtonColumn).
- Set its Name, HeaderText, and other properties.
- Add it to the DataGridView.Columns collection.
- Optionally, fill its cells manually in code.

**Example Code**

```
using System;
```

```
using System.Windows.Forms;
```

```
public class UnboundColumnExample : Form
```

```
{
```

```
    private DataGridView dataGridView1 = new DataGridView();
```

```
    public UnboundColumnExample()
```

```
{
```

```
        dataGridView1.Dock = DockStyle.Fill;
```

```
        Controls.Add(dataGridView1);
```

```
        // Example: Bind to a data source
```

```
dataGridView1.DataSource = new[]
```

```
{  
    new { ID = 1, Name = "Alice" },  
    new { ID = 2, Name = "Bob" }  
};
```

```
// Create an unbound column
```

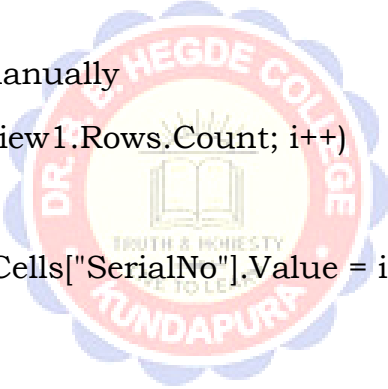
```
DataGridViewTextBoxColumn unboundCol = new DataGridViewTextBoxColumn();  
unboundCol.Name = "SerialNo";  
unboundCol.HeaderText = "Serial No";
```

```
// Add unbound column to DataGridView
```

```
dataGridView1.Columns.Insert(0, unboundCol);
```

```
// Fill unbound column manually
```

```
for (int i = 0; i < dataGridView1.Rows.Count; i++)  
{  
    dataGridView1.Rows[i].Cells["SerialNo"].Value = i + 1;  
}  
}
```



```
[STAThread]
```

```
public static void Main()
```

```
{  
    Application.Run(new UnboundColumnExample());  
}  
}
```

### Explanation

- The column "SerialNo" is unbound because it isn't tied to the data source.
- It is added manually using the Columns.Insert() method.
- Its values are filled programmatically using a loop.

**15. Explain common values used inConnectionString property. (5)**

Common Values Used in ConnectionString Property in .NET

The ConnectionString property in .NET specifies the details required to connect an application to a data source (like SQL Server, Oracle, or Access).

It contains key-value pairs that define server, database, authentication, and other connection settings.

**Common Values (Parameters)**

| Property / Key             | Description                                               | Example                    |
|----------------------------|-----------------------------------------------------------|----------------------------|
| Data Source / Server       | Specifies the name of the database server or instance.    | Data Source=localhost;     |
| Initial Catalog / Database | Defines the name of the database to connect to.           | Initial Catalog=StudentDB; |
| User ID                    | Specifies the username for SQL Server authentication.     | User ID=sa;                |
| Password                   | Specifies the password for the SQL Server login.          | Password=12345;            |
| Integrated Security        | Uses Windows authentication when set to True or SSPI.     | Integrated Security=True;  |
| Provider                   | Specifies the data provider (used in OLE DB connections). | Provider=SQLOLEDB;         |

**Example Connection String**

// Using SQL Server Authentication

```
string connStr = "Data Source=SERVER1;Initial Catalog=StudentDB;User  
ID=sa;Password=12345;";
```

// Using Windows Authentication

```
string connStr = "Data Source=SERVER1;Initial Catalog=StudentDB;Integrated  
Security=True;";
```

**Explanation**

- The connection string contains parameters and values separated by semicolons.
- It is passed to a connection object like SqlConnection to open a link with the database.

Example:

```
SqlConnection con = new SqlConnection(connStr);  
con.Open();
```

**16. Explain the common properties of SqlConnectionStringBuilder. (5)**

The SqlConnectionStringBuilder class in .NET is used to create and manage connection strings for SQL Server in a safe and structured way.

It provides strongly typed properties instead of manually concatenating key-value pairs in a string.

**Common Properties**

| Property               | Description                                                              | Example Value |
|------------------------|--------------------------------------------------------------------------|---------------|
| DataSource             | Specifies the SQL Server name or network address.                        | "localhost"   |
| InitialCatalog         | Specifies the database name to connect to.                               | "StudentDB"   |
| UserID                 | Sets the SQL Server login username.                                      | "sa"          |
| Password               | Sets the login password.                                                 | "12345"       |
| IntegratedSecurity     | Enables Windows Authentication when set to true.                         | true          |
| ConnectTimeout         | Defines the time (in seconds) to wait before connection times out.       | 30            |
| Encrypt                | Indicates whether data is encrypted during transfer.                     | true or false |
| TrustServerCertificate | Determines whether the server certificate is trusted without validation. | true          |

**Example Code**

```
using System;
using System.Data.SqlClient;
class Example
{
    static void Main()
    {
        SqlConnectionStringBuilder builder = new SqlConnectionStringBuilder();

        builder.DataSource = "SERVER1";
        builder.InitialCatalog = "StudentDB";
        builder.UserID = "sa";
        builder.Password = "12345";
        builder.IntegratedSecurity = false;
```

```

string connString = builder.ConnectionString;

Console.WriteLine(connString);

} }

```

### Explanation

- The SqlConnectionStringBuilder automatically formats and validates the connection string.
- It reduces errors caused by typos in manual connection string creation.
- The final connection string is stored in the ConnectionString property.

### 17. Explain common properties and methods of SqlCommand Class. (5)

The SqlCommand class is used to execute SQL queries or stored procedures against a SQL Server database.

It works with a SqlConnection object to perform CRUD operations (Create, Read, Update, Delete).

#### Common Properties

| Property       | Description                                                           |
|----------------|-----------------------------------------------------------------------|
| CommandText    | Holds the SQL query or stored procedure name.                         |
| CommandType    | Specifies the type of command: Text, StoredProcedure, or TableDirect. |
| Connection     | The SqlConnection object associated with this command.                |
| Parameters     | Collection of parameters for parameterized queries.                   |
| CommandTimeout | Time in seconds to wait for query execution before timing out.        |

#### Common Methods

| Method             | Description                                                                            |
|--------------------|----------------------------------------------------------------------------------------|
| ExecuteNonQuery()  | Executes commands like INSERT, UPDATE, DELETE and returns the number of affected rows. |
| ExecuteScalar()    | Executes a query that returns a single value.                                          |
| ExecuteReader()    | Executes a query and returns a SqlDataReader to read multiple rows.                    |
| ExecuteXmlReader() | Executes a query and returns an XmlReader for XML data.                                |
| Cancel()           | Cancels the execution of the command.                                                  |

**Example Code**

```
using System;
using System.Data.SqlClient;
class Example
{
    static void Main()
    {
        SqlConnection con = new SqlConnection("Data Source=SERVER1;Initial
Catalog=StudentDB;Integrated Security=True");
        SqlCommand cmd = new SqlCommand();

        cmd.Connection = con;
        cmd.CommandText = "SELECT COUNT(*) FROM Students";
        cmd.CommandType = System.Data.CommandType.Text;

        con.Open();
        int studentCount = (int)cmd.ExecuteScalar();
        con.Close();

        Console.WriteLine("Total Students: " + studentCount);
    }
}
```

**Explanation**

- CommandText contains the SQL query.
- ExecuteScalar retrieves a single value from the database.
- Parameters can be added to prevent SQL injection.

**18. Explain the characteristics of ASP .NET. (5)**

ASP.NET is a server-side web application framework developed by Microsoft for building dynamic web pages, web applications, and web services.

It provides powerful tools, security, and performance features for modern web development.

**Key Characteristics**

- I. Server-Side Technology

- a. All code executes on the server before the page is sent to the browser.
- b. Ensures better performance, security, and browser independence.

## II. Language Independence

- a. Developers can write code using C#, VB.NET, or other .NET languages.
- b. Supports integration of multiple languages within a single application.

## III. Rich Toolbox and Controls

- a. Provides built-in server controls, data controls, and validation controls.
- b. Simplifies creation of interactive and data-driven web pages.

## IV. State Management

- a. ASP.NET offers mechanisms like ViewState, Session State, and Application State to maintain user data across requests.

## V. Security Features

- a. Built-in authentication and authorization (Forms, Windows, Identity).
- b. Supports request validation and configuration-based security.

## VI. Easy Deployment and Configuration

- a. Configuration through web.config file; supports auto-deployment and side-by-side versioning.

## 19. Explain the architecture of ASP .NET. (5)

### a. Introduction (1 mark)

ASP.NET is a server-side web application framework developed by Microsoft. It follows a three-tier architecture consisting of Presentation Layer, Business Logic Layer, and Data Access Layer, all running on the .NET Framework / .NET runtime environment.

### 2. Main Components / Layers (3 marks)

#### a) Common Language Runtime (CLR)

- Executes ASP.NET applications.
- Handles memory management, security, exception handling, and code execution.
- Supports multiple languages (C#, VB.NET).

#### b) .NET Framework Class Library (FCL)

- A large collection of reusable classes and APIs for web development (e.g., file I/O, database access, XML, etc.).
- Provides built-in support for ADO.NET, ASP.NET controls, and data handling.

#### c) ASP.NET Runtime (Page Framework)

- Processes incoming HTTP requests from clients.

- Converts .aspx pages into server-side classes and renders the output as HTML to the browser.
- Includes HTTP pipeline, Page Lifecycle, and event handling model.

**d) Web Server (IIS - Internet Information Services)**

- Hosts and manages ASP.NET web applications.
- Handles request/response, authentication, and application pool management.

**e) Application Layers**

- Presentation Layer – UI components (Web Forms, Razor pages).
- Business Logic Layer – Application rules and data processing.
- Data Access Layer – Communicates with databases using ADO.NET or Entity Framework.

**f. Request Processing Flow (1 mark)**

1. Client sends a request (e.g., index.aspx) via browser.
2. IIS receives the request and forwards it to the ASP.NET runtime.
3. ASP.NET processes the page, executes server-side code, and interacts with the database if needed.
4. The final HTML output is sent back to the browser.

**20. Write the meaning of the following properties of Web TextBox control (5)**

**a) AutoCompleteType. b) CausesValidation c) MaxLength d) Rows e) TextMode.**

**properties of the ASP.NET Web TextBox control:****a) AutoCompleteType (1 mark)**

Specifies what kind of auto-completion the browser should offer for the TextBox, such as Email, Username, Password, or None.

Example:

```
<asp:TextBox ID="txtEmail" runat="server" AutoCompleteType="Email" />
```

*Helps users by suggesting previously entered values.*

**b) CausesValidation (1 mark)**

Indicates whether validation controls on the page should run when the TextBox causes a postback (e.g., when used with a button).

- True – triggers validation.
- False – bypasses validation.

Example:

```
<asp:TextBox ID="txtName" runat="server" CausesValidation="True" />
```



**c) MaxLength (1 mark)**

Defines the maximum number of characters that a user can enter into the TextBox.  
Example:

```
<asp:TextBox ID="txtPhone" runat="server" MaxLength="10" />
```

**d) Rows (1 mark)**

Specifies the number of visible lines in a multi-line TextBox.  
Works only when TextMode="MultiLine".

Example:

```
<asp:TextBox ID="txtAddress" runat="server" TextMode="MultiLine" Rows="4" />
```

**e) TextMode (1 mark)**

Determines the type of input allowed in the TextBox.  
Possible values:

- SingleLine – default single-line input.
- MultiLine – allows multiple lines of text.
- Password – masks input characters.

Example:

```
<asp:TextBox ID="txtPassword" runat="server" TextMode="Password" />
```

**21. Explain ASP .NET web Button control with its unique properties. (5)****1. Definition (1 mark)**

The **Button control** in ASP.NET is a **server-side control** used to create a clickable button on a web page that performs actions such as submitting a form or triggering an event on the server.

Example:

```
<asp:Button ID="btnSubmit" runat="server" Text="Submit" OnClick="btnSubmit_Click" />
```

**2. Purpose / Functionality (1 mark)**

When the user clicks the button, it **posts the form data to the server** and executes the **Click** event handler defined in the code-behind file.

Example in C#:

```
protected void btnSubmit_Click(object sender, EventArgs e)
{
    lblMessage.Text = "Button clicked!";
}
```

### 3. Unique Properties (3 marks)

| Property                | Description                                                                                     |
|-------------------------|-------------------------------------------------------------------------------------------------|
| <b>Text</b>             | Specifies the caption displayed on the button (e.g., "Submit", "Save").                         |
| <b>CommandName</b>      | Used to specify a command name for buttons inside data-bound controls (e.g., "Edit", "Delete"). |
| <b>CommandArgument</b>  | Passes additional information with the command, useful in controls like GridView or Repeater.   |
| <b>PostBackUrl</b>      | Allows the button to post the form to a different page (cross-page postback).                   |
| <b>CausesValidation</b> | Indicates whether clicking the button triggers page validation (True/False).                    |

### 22.Explain the important properties of BaseValidator Control. (5)

Important Properties of BaseValidator Control

#### 1. ControlToValidate

- Specifies the ID of the input control (like TextBox) that this validator should validate.
- Example:
- `<asp:RequiredFieldValidator ControlToValidate="txtName" ... />`

#### 2. ErrorMessage

- The message displayed when validation fails.
- This text is shown inside the ValidationSummary control or next to the input.
- Example:
- `ErrorMessage="Name is required"`

#### 3. Text

- The short message displayed next to the control when validation fails.
- If not set, ErrorMessage will be shown instead.
- Example:
- `Text="*"`

#### 4. Display

- Defines how the error message is displayed.
- Values:

- Static → takes up space even when hidden.

- Dynamic → occupies space only when an error occurs.
- None → message is not shown inline (used only in ValidationSummary).

#### 5. EnableClientScript

- When set to true (default), validation runs on the client-side using JavaScript.
- When false, validation runs only on the server-side.

#### 6. SetFocusOnError

- If true, when validation fails, the focus moves automatically to the invalid control.

#### 7. Enabled

- Enables or disables the validator.
- If false, the validator does not perform validation.

### 23. Explain RangeValidator and CompareValidator controls. (6)

#### 1. RangeValidator

Purpose:

Ensures the user's input falls within a specified range.

Key Properties:

| Property          | Description                                          |
|-------------------|------------------------------------------------------|
| ControlToValidate | The input control to validate (e.g., a TextBox).     |
| MinimumValue      | The lowest valid value.                              |
| MaximumValue      | The highest valid value.                             |
| Type              | The data type (e.g., Integer, Double, Date, String). |
| ErrorMessage      | The message shown if validation fails.               |

Example:

```
<asp:TextBox ID="txtAge" runat="server"></asp:TextBox>  
<asp:RangeValidator  
  ID="rvAge"  
  runat="server"  
  ControlToValidate="txtAge"  
  MinimumValue="18"
```



```
MaximumValue="60"
```

```
Type="Integer"
```

```
ErrorMessage="Age must be between 18 and 60."
```

```
ForeColor="Red">
```

```
</asp:RangeValidator>
```

Use Case:

- Validating age range.
- Ensuring a number is between 1 and 100.
- Ensuring a date is within a specific window.

## 2. CompareValidator

Purpose:

Compares the value of one control to another control or a fixed value using a comparison operator.

Key Properties:

| Property          | Description                                                            |
|-------------------|------------------------------------------------------------------------|
| ControlToValidate | The control you want to validate.                                      |
| ControlToCompare  | Another control to compare to (e.g., for password confirmation).       |
| ValueToCompare    | A fixed value to compare against (used if not comparing two controls). |
| Operator          | Comparison type: Equal, NotEqual, GreaterThan, LessThan, etc.          |
| Type              | Data type (String, Integer, Date, etc.).                               |
| ErrorMessage      | Message shown on validation failure.                                   |

### Example : Compare Two Controls (e.g., Confirm Password)

```
<asp:TextBox ID="txtPassword" runat="server" TextMode="Password"></asp:TextBox>
```

```
<asp:TextBox ID="txtConfirmPassword" runat="server"  
TextMode="Password"></asp:TextBox>
```

```
<asp:CompareValidator
```

```
  ID="cvPassword"
```

```
  runat="server"
```

```
  ControlToValidate="txtConfirmPassword"
```

```
  ControlToCompare="txtPassword"
```

```
  Operator="Equal"
```

```

Type="String"
ErrorMessage="Passwords do not match."
ForeColor="Red">
</asp:CompareValidator>

```

## 24. Explain important properties of ValidationSummary control. (5)

The ValidationSummary control in ASP.NET Web Forms is used to display a summary of validation errors on a web form. Instead of showing error messages next to each individual control (like RequiredFieldValidator, etc.), it aggregates all error messages in one place.

### Purpose of ValidationSummary

- To display all validation errors in a single centralized block.
- Improves user experience by summarizing issues at the top or bottom of the form.
- Works in tandem with other validation controls (RequiredFieldValidator, RangeValidator, etc.).

### Important Properties of ValidationSummary

| Property           | Description                                                                               |
|--------------------|-------------------------------------------------------------------------------------------|
| ShowSummary        | If true, displays the summary on the page. Usually set to true.                           |
| ShowMessageBox     | If true, shows a JavaScript alert box with validation messages (client-side only).        |
| HeaderText         | Optional heading text shown above the list of errors (e.g., "Please fix the following:"). |
| DisplayMode        | Defines how the error messages are displayed (list, bullet list, or single paragraph).    |
| EnableClientScript | Determines whether client-side script is used (default is true).                          |
| CssClass           | CSS class for styling the summary.                                                        |
| ForeColor          | Sets the text color of the summary.                                                       |

### DisplayMode Options

| Value      | Description                             |
|------------|-----------------------------------------|
| List       | Shows plain list of messages.           |
| BulletList | Adds bullet points before each message. |

| Value | Description |
|-------|-------------|
|-------|-------------|

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| SingleParagraph | Combines all messages into a single paragraph (space-separated). |
|-----------------|------------------------------------------------------------------|

